HOCHSCHULE HAMM-LIPPSTADT

Hardware Engineering Lab

INITIAL PROJECT PROPOSAL

Project Name: FPGA-Based Traffic-Light Controller with Pedestrian Request Button

Team	C3
Prepared by	Md Riajul Islam
Team members	1. Md Riajul Islam 2. Nadia Afruz 3. Sayed Galib Hossen Rizvi 4. Ronjon Sarker
Document type	Initial concept draft with raw engineering sketches
Date	10-06-2026

This document is a raw but complete first proposal. Timing values, FPGA part selection, PCB footprint choices and final state details can be refined after the first team review.

Table of Contents

1. Proposal Summary

This project proposes the design and implementation of an FPGA-based traffic-light controller for a simplified two-direction road intersection. The controller will be described in VHDL, verified through simulation, implemented on an FPGA development board and represented as a custom PCB design. A pedestrian request button will allow a pedestrian to request a safe crossing phase. The request will be stored by the controller and served only after the vehicle traffic has been brought to a safe all-red condition.

The project is intentionally limited to a manageable but realistic scope. It demonstrates finite-state-machine design, counters, clock division, input synchronization, push-button handling, LED output control, FPGA implementation and PCB design without introducing unnecessary protocol or sensor complexity.

Item	Initial proposal
Project title	FPGA-Based Traffic-Light Controller with Pedestrian Request Button
Core technology	VHDL, FPGA implementation, custom PCB design
Prototype platform	Nexys A7 FPGA board for the first functional validation
Main user input	Pedestrian request push button
Main outputs	North-South and East-West vehicle traffic LEDs, pedestrian WAIT/WALK LEDs, optional request status LED
Main design method	Synchronous finite-state machine with configurable phase timers
PCB approach	Project-specific evaluation board or peripheral interface board; final FPGA selection after resource estimation

2. Initial Understanding and Assumptions

The following statements define the first engineering interpretation of the selected project. They are deliberately written as assumptions so that the team can confirm or revise them during the concept-review meeting.

- The system represents one road intersection with two vehicle directions: North-South (NS) and East-West (EW).
- Each vehicle direction has three LED outputs: red, yellow and green.
- The pedestrian crossing is represented by two LED outputs: WAIT and WALK.
- The pedestrian request is generated using one push button. The request is stored until it can be served safely.
- Pedestrian WALK is never active while any vehicle green or yellow light is active.
- An all-red safety interval is inserted between conflicting traffic phases.
- The controller operates from the FPGA board clock. A clock-divider or tick-generator module creates a human-visible timing reference.
- Timing values are configurable VHDL constants so that simulation can run quickly while FPGA demonstration uses longer visible durations.
- The first prototype will be implemented on the Nexys A7 board. The PCB will contain only the hardware required by this project, based on the final design choice.

Important scope decision for the first version: the project does not need real road sensors, wireless communication, VGA graphics or a microprocessor. The push button represents the pedestrian request input.

3. Problem Statement

A traffic-light controller must coordinate mutually exclusive vehicle movements and provide a safe pedestrian crossing interval. The main engineering challenge is not only to switch LEDs in a sequence, but also to guarantee that conflicting

directions are never released at the same time. The controller must accept an asynchronous human button press, store the request and schedule a pedestrian phase without violating the vehicle-light safety sequence.

The proposed design solves this problem with a deterministic synchronous finite-state machine. Every output combination is associated with a defined state. Transitions occur only when the relevant timer expires or when a stored pedestrian request must be served.

4. Project Objectives

1. Design a clear state-based traffic-control algorithm for a two-direction intersection.
2. Implement the complete controller in synthesizable VHDL.
3. Implement a pedestrian-request mechanism with synchronization, debouncing and request storage.
4. Generate timing signals from the FPGA clock using a reusable clock-divider or tick-generator module.
5. Create VHDL testbenches that verify normal sequencing, pedestrian-request behavior, reset handling and safety conditions.
6. Synthesize, implement and program the design on the Nexys A7 FPGA board.
7. Prepare a project-specific PCB schematic, layout concept, 3D view, BOM and manufacturing outputs in Altium Designer.
8. Document the team workflow, design decisions, verification evidence and individual contributions.

5. Functional Requirements

5.1 Mandatory requirements

ID	Requirement	Acceptance criterion
FR-01	The NS direction shall have RED, YELLOW and GREEN outputs.	Exactly one NS vehicle color is active in each normal phase.
FR-02	The EW direction shall have RED, YELLOW and GREEN outputs.	Exactly one EW vehicle color is active in each normal phase.
FR-03	NS and EW GREEN outputs shall never be active simultaneously.	The testbench contains a safety assertion for this condition.
FR-04	The controller shall include a pedestrian request push button.	A button press sets a stored request flag.
FR-05	The request shall be served only in a safe all-red vehicle condition.	Pedestrian WALK becomes active only after vehicle directions are RED.
FR-06	The controller shall include a reset input.	Reset returns the controller to a defined all-red startup state.
FR-07	The controller shall use configurable timing constants.	Simulation values can be reduced without redesigning the FSM.
FR-08	The design shall be synthesizable and implemented on the FPGA board.	Vivado synthesis, implementation and bitstream generation complete successfully.

5.2 Optional enhancements after the mandatory version works

- Blinking pedestrian WAIT LED during the pedestrian-clearance interval.
- Request-accepted status LED to show that the button press has been stored.
- Manual demonstration mode with shorter traffic phases.

- Night mode with flashing yellow output.
- Vehicle-request buttons that simulate simple road sensors.
- Seven-segment countdown display for the remaining pedestrian crossing time.

6. Proposed System Architecture

The initial architecture is divided into small VHDL modules so that each block can be implemented and tested independently. The top-level architecture is shown in the raw sketch below.

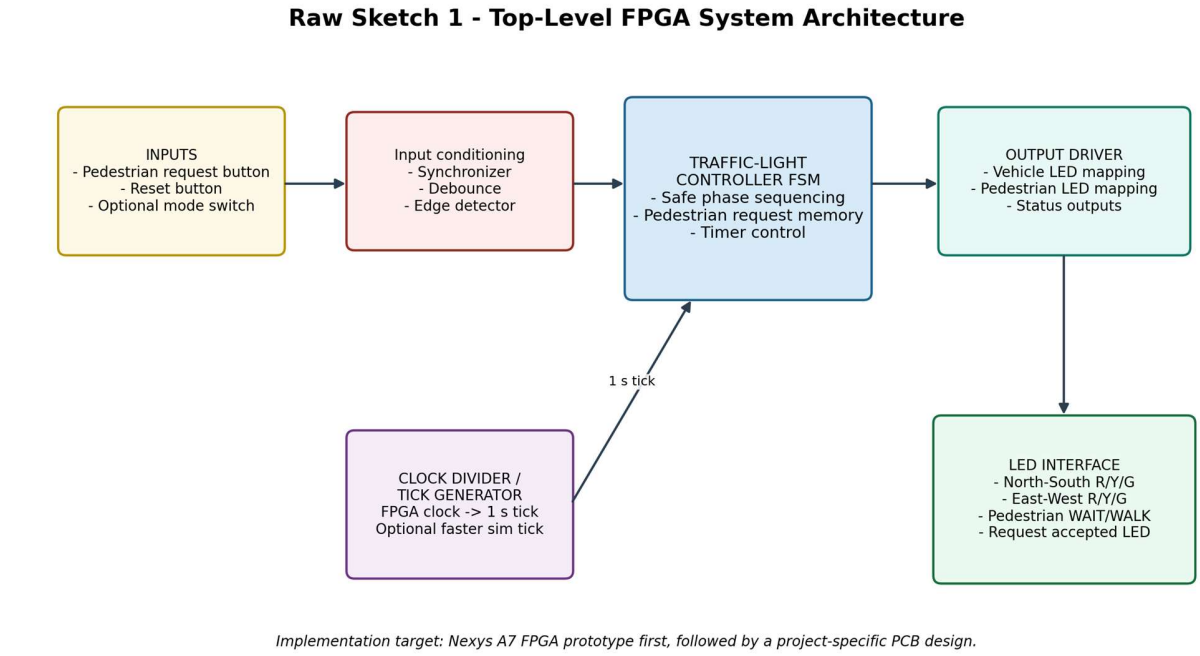


Figure 1. Raw top-level architecture sketch for the proposed traffic-light controller.

Block	Purpose
Input synchronizer	Reduces metastability risk when asynchronous push buttons enter the FPGA clock domain.
Debouncer and edge detector	Prevents one physical button press from being interpreted as multiple requests.
Tick generator	Generates a slow periodic enable pulse from the FPGA board clock.
Traffic-light FSM	Controls safe sequencing of NS, EW and pedestrian phases.
Timer counter	Measures the duration of each active state using the generated tick.
Output mapper	Converts FSM states into LED control signals.
Top-level entity	Connects modules and exposes board pins for the XDC constraints file.

7. Traffic-Light Operating Concept

7.1 Initial operating phases

State	Vehicle outputs	Pedestrian outputs	Transition condition
S0 - Reset / startup	NS RED, EW RED	WAIT active	Startup safety timer expires
S1 - NS green	NS GREEN, EW RED	WAIT active	NS green timer expires
S2 - NS yellow	NS YELLOW, EW RED	WAIT active	Yellow timer expires
S3 - All red clearance	NS RED, EW RED	WAIT active	Clearance timer expires; branch according to request policy
S4 - EW green	NS RED, EW GREEN	WAIT active	EW green timer expires
S5 - EW yellow	NS RED, EW YELLOW	WAIT active	Yellow timer expires
S6 - All red clearance	NS RED, EW RED	WAIT active	If request pending, start pedestrian WALK; otherwise resume vehicle cycle
S7 - Pedestrian walk	NS RED, EW RED	WALK active	Walk timer expires
S8 - Pedestrian clearance	NS RED, EW RED	WAIT active or optional blink	Clearance timer expires

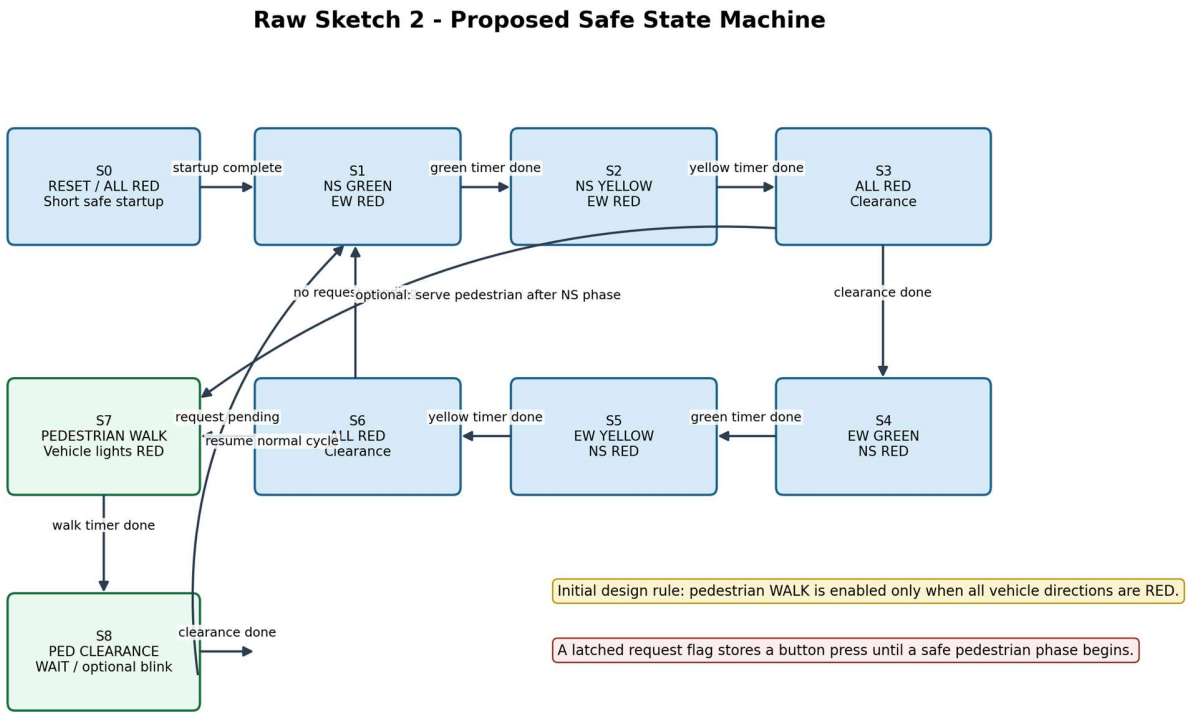


Figure 2. Preliminary safe-state sequence. The exact pedestrian insertion point can be adjusted after team discussion.

7.2 Pedestrian request policy

When the pedestrian request button is pressed, the design sets an internal request_pending flag. The flag remains set until the FSM enters the pedestrian WALK state. This ensures that a short button press is not lost. The request does not immediately interrupt a green or yellow vehicle phase. Instead, the FSM completes the current safe sequence, enters an all-red condition and then activates pedestrian WALK.

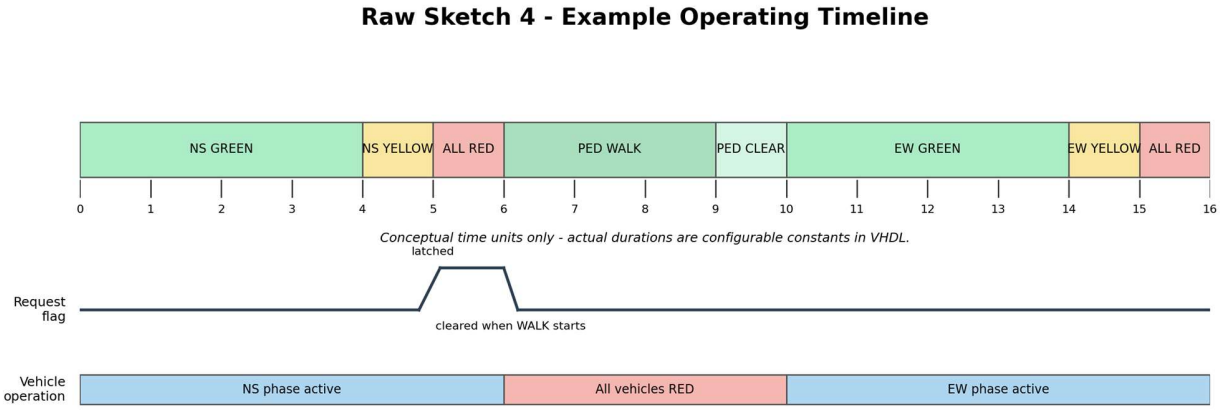


Figure 3. Example timeline showing how a request is stored and served at a safe point.

8. Proposed VHDL Design Structure

The VHDL files should be kept modular and readable. The following structure is proposed for the initial implementation.

File name	Entity or package	Responsibility
traffic_light_pkg.vhd	traffic_light_pkg	State type declarations, timing constants and reusable definitions.
tick_generator.vhd	tick_generator	Create a slow timing tick from the FPGA clock.
button_conditioner.vhd	button_conditioner	Synchronize, debounce and detect a button press edge.
traffic_light_fsm.vhd	traffic_light_fsm	Main sequential state register, timer handling and transition logic.
traffic_light_output_decoder.vhd	traffic_light_output_decoder	Map FSM states to NS, EW and pedestrian LED outputs.
traffic_light_top.vhd	traffic_light_top	Top-level integration for the Nexys A7 board.
traffic_light_top.xdc	-	FPGA pin assignments and clock constraint.
traffic_light_fsm_tb.vhd	traffic_light_fsm_tb	FSM verification with reduced simulation timing values.
traffic_light_top_tb.vhd	traffic_light_top_tb	Integration-level verification of button request and output sequencing.

8.1 Suggested internal signals

- clk: FPGA board clock input.

- rst: synchronous or asynchronous reset, to be selected consistently across the design.
- tick_1s: slow enable pulse generated by the tick generator.
- ped_button_raw: external pedestrian button input.
- ped_request_pulse: debounced one-clock pulse.
- request_pending: stored request flag.
- state_current and state_next: finite-state-machine state signals.
- phase_counter: counter for the active phase duration.
- ns_red, ns_yellow, ns_green, ew_red, ew_yellow, ew_green: vehicle LED outputs.
- ped_wait, ped_walk: pedestrian LED outputs.

9. FPGA Prototype Plan

The first functional prototype will be implemented on the Nexys A7 FPGA development board. The board provides switches, push buttons and LEDs that are suitable for validating the logic before the PCB design is finalized.

Prototype item	Initial mapping concept
Clock input	Use the onboard Nexys A7 system clock and generate a slower timing tick in VHDL.
Reset	Use one onboard push button.
Pedestrian request	Use one onboard push button.
NS lights	Use three onboard LEDs: RED, YELLOW and GREEN.
EW lights	Use three onboard LEDs: RED, YELLOW and GREEN.
Pedestrian lights	Use two onboard LEDs: WAIT and WALK.
Request status	Use one optional onboard LED.
Constraints file	Create an XDC file after selecting the exact Nexys A7 pins.

9.1 FPGA implementation evidence to collect

- Simulation screenshots showing normal sequencing and pedestrian-request handling.
- Vivado synthesis report with logic utilization.
- Implementation report and timing summary.
- User constraints file.
- Generated bitstream.
- Photos or a short demonstration video of the programmed FPGA board.

10. PCB Design Concept

The PCB stage will translate the working FPGA concept into a dedicated hardware design. At this initial stage, the exact FPGA and board architecture are intentionally left open. The team should first estimate the required FPGA resources, select a manageable FPGA device or compact FPGA module, and then include only the peripherals required by the traffic-light project.

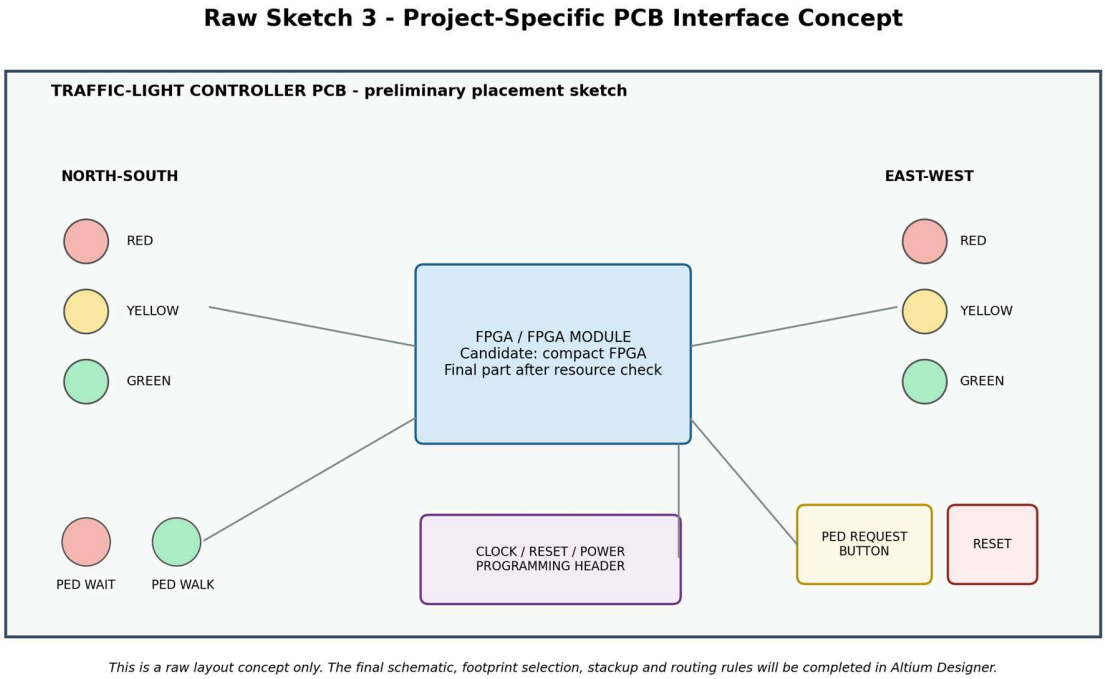


Figure 4. Raw PCB interface and placement concept. This is not a routed PCB layout.

10.1 Initial PCB blocks

PCB block	Purpose	Initial components
FPGA processing	Run the VHDL controller	Compact FPGA or FPGA module, configuration interface
Power supply	Provide stable rails	Input connector, regulator(s), decoupling capacitors, power LED
Clock and reset	Provide timing and initialization	Oscillator if required, reset push button, pull-up or pull-down resistor
Vehicle indicators	Represent traffic lights	Six LEDs and current-limiting resistors
Pedestrian indicators	Represent WAIT and WALK	Two LEDs and current-limiting resistors
Pedestrian input	Accept crossing request	Push button, pull resistor, optional RC filtering
Programming interface	Allow FPGA configuration	Programming header or USB interface depending on FPGA choice
Debug expansion	Support testing	Test points or small header for key signals

10.2 PCB outputs required for the final submission

- Schematic pages with clear net labels and annotations.
- Electrical Rules Check (ERC) result.
- PCB layout with component placement and routing.
- Design Rules Check (DRC) result.
- Physical stackup and routing constraints.
- 3D PCB view.

- Bill of Materials (BOM).
- Gerber and fabrication files.

11. Verification and Validation Plan

Verification must demonstrate both functional behavior and safety. A traffic-light design is simple enough to test thoroughly, which makes it suitable for the lab project.

Test ID	Scenario	Expected result
TB-01	Apply reset.	The controller enters startup all-red state.
TB-02	Allow the normal cycle to run without a pedestrian request.	NS and EW phases alternate with yellow and all-red clearance intervals.
TB-03	Press the pedestrian button during NS green.	The request is stored but WALK does not begin immediately.
TB-04	Reach a safe all-red phase with request_pending = 1.	The FSM enters pedestrian WALK and clears the stored request.
TB-05	Press the pedestrian button more than once before service.	Only one pending request is stored; the controller remains stable.
TB-06	Press reset from each operating state.	The system returns to the defined reset state.
TB-07	Check safety assertion continuously.	NS GREEN and EW GREEN are never active together.
TB-08	Check pedestrian safety assertion continuously.	ped_walk is never active while a vehicle green or yellow output is active.
TB-09	Run implementation on the FPGA board.	LED behavior matches the simulated sequence.

11.1 Recommended safety assertions in the testbench

- Assert that not (ns_green = 1 and ew_green = 1).
- Assert that pedestrian WALK implies ns_red = 1 and ew_red = 1.
- Assert that each vehicle direction has only one active color at a time.
- Assert that reset produces the expected safe state.

12. Work Packages and Team Division

The table below is an initial task-allocation sketch. The names can be updated after the team discussion. Each member should document individual contributions because both teamwork and personal work are relevant to the project assessment.

Work package	Main tasks	Responsible person
WP1 - Requirements and FSM	Confirm scope, define states, create transition table, define timing constants	Md Riajul Islam / [confirm]
WP2 - VHDL timing and input modules	Implement tick generator, synchronizer, debounce and edge detector	[Team member 2]
WP3 - VHDL FSM and output mapping	Implement controller FSM, timer logic and LED decoder	[Team member 3]

Work package	Main tasks	Responsible person
WP4 - Testbench and FPGA implementation	Create simulation scenarios, assertions, XDC file, synthesis and bitstream	Shared; assign lead
WP5 - PCB schematic	Select FPGA approach, power circuit, LEDs, buttons, programming interface	Shared; assign PCB lead
WP6 - PCB layout and manufacturing files	Placement, routing, ERC/DRC, 3D view, BOM and Gerbers	Shared; assign PCB lead
WP7 - Documentation and presentation	Concept draft, final report, figures, screenshots and presentation	All members

13. Expected Deliverables

13.1 Concept-draft deliverables

- Project title and team-member list.
- Concept description and problem statement.
- Requirements and scope assumptions.
- System block diagram.
- Initial FSM diagram.
- Planned peripherals and PCB concept.
- Task division and open questions.

13.2 Final VHDL and FPGA deliverables

- VHDL source files and testbenches.
- Vivado project or reproducible source package.
- XDC constraints file.
- Simulation, synthesis and implementation evidence.
- Bitstream file.
- FPGA board demonstration evidence.

13.3 Final PCB deliverables

- Altium PCB project.
- Schematic pages.
- PCB layout.
- ERC and DRC evidence.
- 3D PCB view.
- BOM.
- Gerber and fabrication outputs.

14. Risks, Open Questions and Decisions

Topic	Initial position	Decision required
Pedestrian insertion policy	Serve a request after the current vehicle sequence reaches a safe all-red state.	Confirm whether the team wants service after every full cycle or at the earliest safe clearance state.

Topic	Initial position	Decision required
Timing durations	Use configurable constants. Start with short simulation values and visible FPGA values.	Choose final green, yellow, walk and clearance durations.
PCB architecture	Design only the required hardware.	Choose between a compact standalone FPGA board and a simpler FPGA-module carrier or peripheral board.
FPGA choice	Nexys A7 for prototype; smaller device may be suitable for PCB.	Estimate resources after VHDL synthesis and select the PCB FPGA solution.
Button conditioning	Implement synchronizer and debounce logic.	Confirm whether hardware RC filtering is also desired on the PCB.
Optional features	Keep mandatory design small.	Select at most one optional feature after the base version passes verification.

15. Preliminary Schedule

Stage	Main output	Suggested sequence
1. Concept confirmation	Approved scope, requirements and FSM sketch	Immediately after team discussion
2. Base VHDL implementation	Tick generator, input conditioning, FSM and output mapper	Start with mandatory functions only
3. Testbench verification	Normal cycle, request handling, reset and safety assertions	Before PCB finalization
4. Nexys A7 implementation	XDC file, synthesis, implementation and bitstream	After simulation is stable
5. PCB architecture decision	Final FPGA approach and components	After resource estimation
6. Altium schematic and layout	ERC, placement, routing and DRC	Iterative
7. Final evidence package	3D view, BOM, Gerbers, screenshots and report	Before final submission
8. Presentation preparation	Short demo and team presentation	After main results are available

16. Initial Conclusion

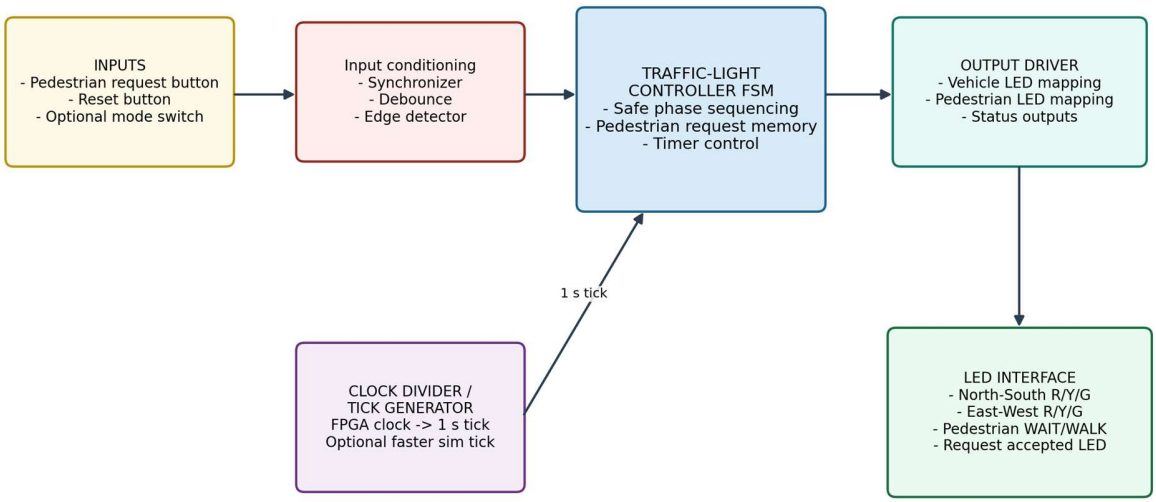
The selected traffic-light controller is an appropriate Hardware Engineering Lab project because it combines a clear real-world use case with manageable digital-design complexity. It requires finite-state-machine reasoning, counters, clock division, asynchronous-input handling, simulation, FPGA implementation and a custom PCB design. The project can be completed in stages: first a safe mandatory controller, then one optional enhancement only if time remains.

The next team action is to approve the intersection model, confirm the pedestrian insertion policy, assign work packages and begin the mandatory VHDL modules.

Appendix A. Raw Sketches for Team Discussion

The following sketches are intentionally simple. They are included to support discussion and can be revised after the team confirms the final design decisions.

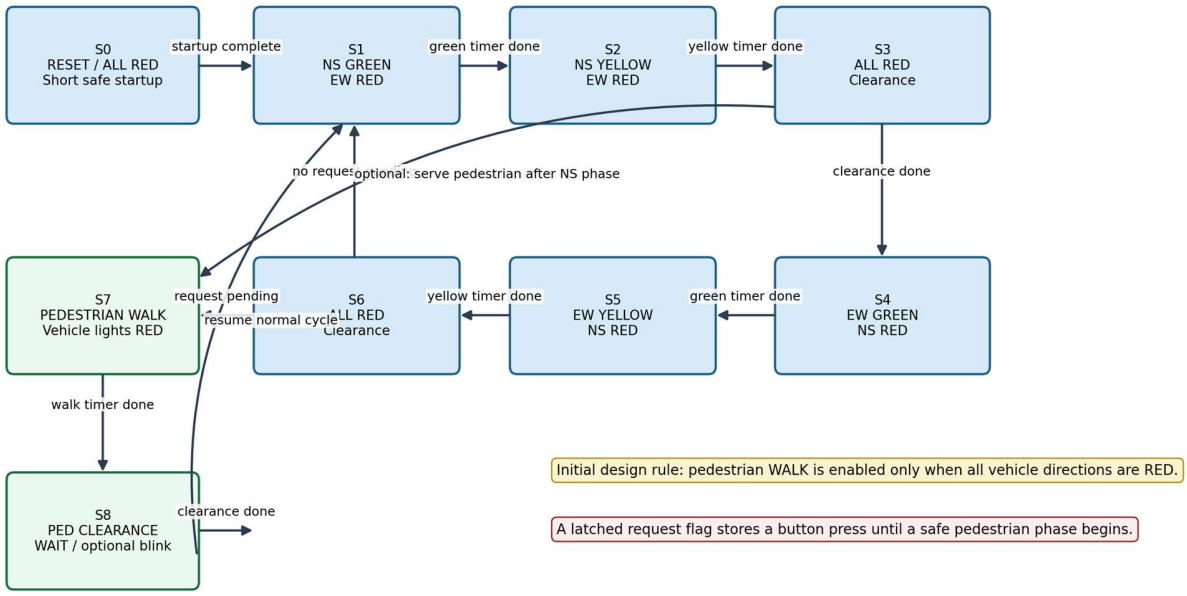
Raw Sketch 1 - Top-Level FPGA System Architecture



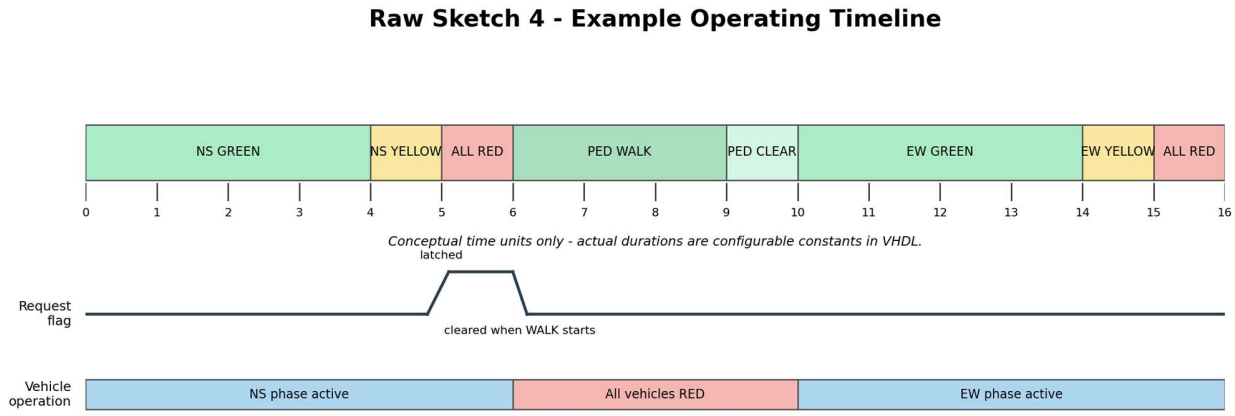
Implementation target: Nexys A7 FPGA prototype first, followed by a project-specific PCB design.

Appendix Figure A1. Top-level system architecture.

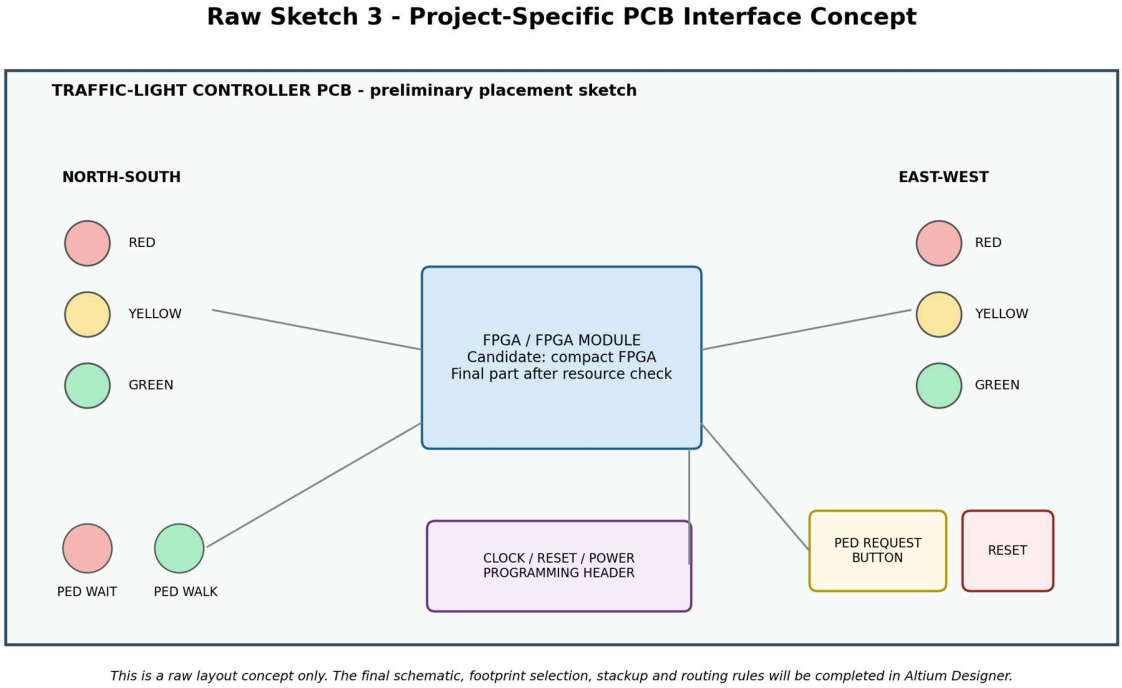
Raw Sketch 2 - Proposed Safe State Machine



Appendix Figure A2. Preliminary state-machine concept.



Appendix Figure A3. Example request-handling timeline.



Appendix Figure A4. Preliminary PCB interface placement sketch.

Appendix B. Team Review Checklist

Question	Team decision / notes
Are NS and EW the correct vehicle directions for the first version?	[Fill during meeting]
Should pedestrian service occur at the earliest safe all-red point or after a complete vehicle cycle?	[Fill during meeting]
What timing values should be demonstrated on the FPGA board?	[Fill during meeting]
Which optional feature, if any, should be added after the mandatory design works?	[Fill during meeting]
Who is responsible for each work package?	[Fill during meeting]
Which PCB architecture will be used?	[Fill after synthesis resource estimate]

Preparation note: This proposal follows the Hardware Engineering Lab project-guidance workflow: concept description, requirements, block diagram, VHDL implementation, testbench verification, FPGA implementation evidence and PCB outputs such as schematic, layout, 3D view, BOM and Gerber files.